Neural Approaches to Code Generation: From Natural Language Understanding to Structured Program Synthesis

Md Mubin Islam Alif

April 14, 2026

Abstract

Code generation nowadays is producing executable code from natural language descriptions. This paper explores the latest neural approaches to code generation, focusing on how models understand natural language and synthesize structured programs. This paper is reviewing three papers: CodeT5 [1], InCoder [2], and StructCoder [3]. Here I am discussing the methodologies, strengths, and limitations of each approach, and how they contribute to advancing the state of the art in code generation. Here CodeT5 presents an identifier-aware encoder-decoder architecture for code understanding and generation [1]. InCoder focuses on code infilling and synthesis through a generative modeling approach [2]. StructCoder adds structure awareness to improve generation quality on code translation and text-to-code tasks [3].

1 Introduction

The idea of code generation from natural language has been fascinating researchers for a long time. With the progressive work in deep learning and some transformer-based models, we have seen significant advancements in this area. The ability of machines to understand natural language and generate structured code has the potential to revolutionize software development. But the issue has been that while models can generate code, they often struggle with understanding the gist of programming logic and structure, which eventually leads to code that may be syntactically correct but semantically flawed. But sometimes even syntactical correctness is flawed as the models are not even aware of the identifiers and structure of the code. This is where models like CodeT5, InCoder, and StructCoder come into play, each addressing different aspects of the code generation problem.

These three models have introduced three different mechanisms to improve code generation. CodeT5 is built on the T5 architecture which is a transformer-based model that has been adapted for code understanding and generation. But on top of it, CodeT5 incorporated the idea of being identifier-aware, which means that it can better understand the role of identifiers in code, which is crucial for generating meaningful and functional code. InCoder, on the other hand, greatly focuses on code infilling and synthesis. It uses a generative modeling approach which allows it to fill in missing parts of code and generate new code based on the context provided. This is particularly useful for tasks like code

completion and code repair. StructCoder went one more step beyond. It added structure awareness to the model, which means that it can better understand the hierarchical and logical structure of the code. These all three models have shown significant improvements in code generation, but they also have their own limitations.

2 Literature Review

2.1 Background and Problem Formulation

When we talk about code generation, we are essentially meaning two things: (1) *natural language to code* (NL-to-PL) generation, where the model takes a natural language description and generates code. (2) *code to natural language* (PL-to-NL) generation, where the model takes code and generates a natural language description of what the code does. Both of these tasks are important for different aspects. The former is crucial for automating software development, where developers can simply describe what they want in natural language and the model can interpret that and generate the corresponding code. The latter is important for code documentation and understanding, where developers use natural language to explain what code does. It helps in understanding the codebase and maintaining the codebase effectively. It eventually helps in documentation and code review processes. But both tasks need to understand the intent, generate syntactically correct code, and also generate semantically meaningful code. This is where the challenge lies. Early code generation focused on neural networks that were trained on large code corpora. But the transformer's self-attention mechanism proved to be much more of a game changer for capturing long-range dependencies in code [4]. Pre-training on large unlabeled code corpora (e.g., GitHub) became standard, with objectives such as masked language modeling (MLM), denoising autoencoding (DAE), and replaced token detection (RTD).

2.2 Key Challenges in Neural Code Generation

Based on the three papers, we can identify several key challenges in natural code generation.

- **Understanding Natural Language**: One of the big challenges out there is that the model needs the ability to understand natural language descriptions accurately. But the thing is that natural language can be ambiguous. The model needs to understand the intent behind the natural language description.

- **Identifier Semantics**: As highlighted by CodeT5, understanding identifiers and their semantics is crucial for generating meaningful code. Identifiers are the names given to variables, functions, classes, etc. If the model does not understand the semantics of identifiers, it may generate code that might be syntactically correct but semantically meaningless.

- **Structural Awareness**: From StructCoder [3], we can see that if we understand the structure of the code, then we can generate better code. Code has a hierarchical structure, and understanding this structure is crucial for generating code that is not only syntactically correct but also semantically meaningful. Here the models need

to understand how the different parts of the code relate to each other and how they fit together.

- **Infilling and Editing**: InCoder [2] focuses on code infilling and synthesis. This is a huge challenge as the models need to be able to fill in missing parts of code and generate new code based on the context provided. This feature is particularly useful for tasks like code completion and code repair, but it requires the model to have a deep understanding of the code context and also the ability to generate code that fits well within that existing context.

- **Bidirectional Alignment**: Another challenge is bidirectional alignment. This means that the model should be well aligned in both directions: from natural language to code and from code to natural language.

2.3 Related Pre-trained Models for Code

Several pre-trained models have been proposed other than the three focal papers I would be discussing. CodeBERT [5] is an encoder-only model trained with MLM and replaced token detection on NL-PL pairs. GraphCodeBERT [6] incorporates data flow graphs into the encoder. PLBART [7] is an encoder-decoder model pre-trained with denoising autoencoding on code and natural language. Even though these models are somewhat effective, the decoder here lacks awareness of code structure.

3 Methodologies

3.1 CodeT5: Identifier-aware Encoder-Decoder Model

CodeT5 is built on the T5 architecture which is a transformer-based model which has been used for various natural language processing tasks (e.g, Code Understanding, code generation, code translation, etc.). Firstly if I want to summarize the architecture of CodeT5 in two words then it would be Explicit and Focused understanding. CodeT5 is designed to be identifier-aware, which means that it understands the role of identifiers in code generally in a better way than other compatriots. For natural language processing tasks, there were several powerful models like BERT, GPT, T5, etc. These models were pre-trained on large corpora of text data and then for smaller tasks they were fine tuned. As a result, even for less amount of data, there was good performance. So researchers were thinking that if we could do the same thing for code, then we could also get good performance for code generation and code understanding tasks. So they were trying to apply NLP models. Example:

```
def add(a, b):
    return a + b
```

Then the model can learn that it is a function and then perform addition. But the main problem occurred that the model seemed to be one-sided. Some models like BERT were just better for understanding the code, and some models acted like just decoders (e.g. GPT), which were just better for generating code. But we wanted both generation and understanding. So these single-type models are not enough for us. The second problem was that the previous models were just taking the words as a sequence. But in reality, code is much more structured. Example:

```
binarySearch(arr.left,right,x)
```

Here the binary search is a function name and those arr, left, right, x are variables. So in CodeT5, they proposed a unified model where the encoder and decoder are trained together. And the main idea around CodeT5 is:

- **Identifier-aware Learning**: The model would understand which one is an identifier and which one is not. Example:

  ```
  def binarySearch(arr.left,right,x)
  ```

  Here the model would understand that binarySearch is a function name and arr is an array and left, right are position variables. Understanding these meanings is necessary.

- **Trained using Masked Language Modeling**: It means that when training, some identifiers are masked and the model is trained to predict those masked identifiers. This helps the model to learn the semantics of identifiers and how they relate to each other in the code.

- **Using code and comment together**: For a lot of code there are comments. Example:

  ```
  # increment value
  def inc(x):
      return x + 1
  ```

  Here the model would learn comment-to-code relation. So if we give the input phrase "increment value", then the model would understand that it needs to generate a function that increments a value. The vice versa is also true.

- **Dual learning**: The model is trained in a dual learning manner, which means it is trained for NL-to-PL generation and PL-to-NL generation at the same time.

The previous models would see the code as a sequence of tokens. Code snippet:

```
def add(a, b):
    return a + b
```

Here the previous models would just see it as a sequence of tokens such as `def`, `add`, `a`, and `b`, along with punctuation symbols. But the problem around that was that all models being treated as the same. But in code all tokens are not equal. Here `def` is a keyword and `add`, `a`, and `b` are identifiers. The identifiers being so much important are because with those developer gives their code a meaning and other people then would understand what the code is trying to interpret and deliver.

The issues of previous models are:

- **Identifier Recognition Issue**: Previously, models would not recognize function names and array names. They would just see them as some random words.

- **Lost of Meaning**: Code snippet:

  ```
  def binarySearch(arr.left,right,x)
  ```

Before, the model would think that binarySearch here is just a token. But in reality, it is a search algorithm, which they would not understand.

- **Problem in Masking**: Code Snippet:

```
def Mask(arr, left, right, x):
```

The previous models would just guess randomly as they would not know that they are identifiers.

Then in CodeT5:

- **Detecting Identifier**: Here it is explicitly trained to detect identifiers. So it would understand that binarySearch is a function name and arr is an array and left, right are position variables. Understanding these meanings is necessary.
- **Specialized Training**:
  - **Task 1: Identifier Tagging**:
    - `arr` → 1 (identifier)
    - `def` → 0 (not identifier)
  - **Task 2: Masked Identifier Prediction**: Code Snippet:

```
def Mask(arr, left, right, x):
```

The model is trained to predict masked identifiers. So it would learn the semantics of identifiers and how they relate to each other in the code. So this model would have to recover binarySearch from the masked code snippet.

The reason behind CodeT5 using T5 architecture is that using one model here, code summarization, generation, translation, and defect detection can be done. So it is a unified model for all code related tasks. The main idea here is that the encoder and decoder are trained together. So the model can learn to understand the code and also generate the code at the same time. T5 uses an Encoder-Decoder structure. There, the encoder understands the input (code/text) and the decoder generates the output. But in CodeT5, it not only understands the code but also generates the code and understands the identifiers. So CodeT5 added identifier awareness and special training objectives to the T5 architecture to improve code understanding and generation performance. The main ideas of pretraining in CodeT5 are:

- **Identifier-aware Masked Span Prediction**: Example 1:

```
def add(a, b):
    return a + b
```

After Masking the code snippet:

```
def [MASK](a, b):
    return a + b
```

Here identifiers are masked more and more as the identifiers are meaningful and important. Example 2:

```
def calculateTotal(price):
```

After Masking the code snippet:

```
def <mask>(<mask>):
```

Here the model would guess calculateTotal and price. So the model gets compelled to learn the context and semantics.

- **Bimodal Dual Generation**: Here the model is trained in two ways:
  - **(A) Natural language to code generation Input (Example):**

    ```
    Add two numbers
    ```

    Output:

    ```
    def add(a, b):
        return a + b
    ```

  - **(B) Code to natural language generation Input (Example):**

    ```
    def add(a, b):
        return a + b
    ```

    Output:

    ```
    Add two numbers
    ```

  As a result, the model understands the code and also it can write the code better.

- **Multi-task Learning**: Here all tasks are trained together. As a result, the model becomes much more versatile and can perform all the tasks using one model.

3.1.1 Model Architecture Overview

The architecture of CodeT5 is based on the T5 model, which is an encoder-decoder transformer architecture. The encoder takes the input and decoder generates the output. The main novelty here is that the model is trained to be identifier-aware.

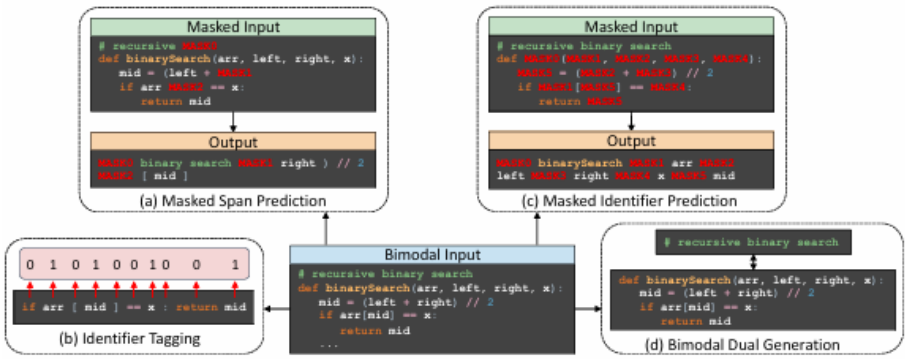


Figure 1: Pre-training tasks of CodeT5. We first alternately train span prediction, identifier prediction, and identifier tagging on both unimodal and bimodal data, and then leverage the bimodal data for dual generation training [1]

The important thing here is its Masked Span Prediction. Here the identifiers are masked more frequently than non-identifiers. Here bimodal dual generation is also important. Here the model is trained to generate code from natural language and also to generate natural language from code. This dual training helps the model to understand the code better and also to generate better code.

3.2 InCoder: Generative Model for Code Infilling and Synthesis

InCoder mainly introduces a generative modeling approach for code infilling and synthesis. The main idea is that the model can fill in missing parts of code and generate new code based on the context provided. For previous models like GPT/Codex, it would just generate code from left to right. But in InCoder, it can generate code in any direction. Now I will discuss the training idea here: Suppose the original code snippet is:

```
def add(a, b):
    return a + b
```

- **Step 1: Masking a little portion**:

```
def add(a, b):
    <MASK>
```

- **Step 2: Taking the hidden part at last**:

```
def add(a, b):
    <MASK>
<MASK> return a + b <EOM>
```

  Now the model is trained here to predict the sequence. As a result, the model learns that there is code at the top, at the bottom there is context, and then what would be the code in the middle. So it learns to fill in the missing part. It can be generalized as such that Left context + Right context → Missing part. So the model learns to generate code based on the context provided. This is bidirectional context and it is very useful for code infilling and synthesis.

Inference in Real: Suppose the code snippet is:

```
def square(x):
    <MASK>
```

Now we give it to the model. Then the model can predict the missing part. And the model gives the output as:

```
    return x * x
```

Real use cases according to the paper [2] are:

- **Variable Rename**: Before:

```
x = 4
y = x + 2
```

  Masked code:

```
<MASK> = 4
y = <MASK> + 2
```

Model:

```
num = 4
y = num + 2
```

- **Docstring Generation**:

```
def add(a, b):
    return a + b
```

Masked code:

```
def add(a, b):
    """<MASK>"""
    return a + b
```

- **Type Prediction**:

```
def add(a: int, b: int) -> <MASK>:
    return a + b
```

The output would be:

```
    int
```

- **Missing Code Fill**:

```
for i in range(n):
    <MASK>
```

The output would be:

```
print(i)
```

The reason this model is this much powerful is that it can understand the context from both sides. So then it is able to generate code that fits very well within the existing context. For old models, it was: only Left → Right. But in InCoder, it is left context + right context → Middle Fill. So it is much more powerful and useful for code infilling and synthesis. The intuitive idea here is that suppose we are solving a problem and writing code:

```
    for(int i = 0; i < n; i++){
        //something missing
    }
```

InCoder basically sees the problem, loop, and context and then tries to guess what should be inside. Suppose we have a code snippet:

```
def add(a, b):
    return a + b
```

Suppose we are trying to mask:

- Left context: `def add (a, b):`
- Right context: `‘‘ ’’`
- Span = `return a + b`

Training Objective [2]

$$\log P\left([\text{Left}; \langle\text{Mask:0}\rangle; \text{Right}; \langle\text{Mask:0}\rangle; \text{Span}; \langle\text{EOM}\rangle]\right) \tag{1}$$

Code snippet:

```
def add(a, b):
    <MASK>
<MASK> return a + b <EOM>
```

Here first `<MASK>` means that something is missing. Then the second `<MASK>` means that generate the code in the middle. And lastly `<EOM>` means that the code is ended.So the model here is learning that there is code at both let and right and trying to predict the missing span. It is kind of learning substring reconstruction problem.

Inference (Real Use) [2]

$$P\left(\cdot \mid [\text{Left}; \langle\text{Mask:0}\rangle; \text{Right}; \langle\text{Mask:0}\rangle]\right) \tag{2}$$

Code snippet:

```
def add(a, b):
    <MASK>
```

Model internally takes it as Left + Mask + Right + Mask Then it generates the code in the middle and gives the output as:

```
    return a + b
```

Full Flow: Training time: Original code → mask → move span to end → train Inference time: Give code with `<MASK>` → model generates the missing part

Example (Multi-region):

```
def process(x):
    <MASK1>
    y = x * 2
    <MASK2>
```

Here the model can fill:

```
<MASK1> print(x)
<MASK2> return y
```

In the previous models:

$$P(\text{token} \mid \text{previous tokens}) \tag{3}$$

In InCoder:

$$P(\text{token} \mid \text{left context} + \text{right context}) \tag{4}$$

It basically means that it is not sequence to sequence. Rather it is autoregressive model plus smart training trick. So it means that it is not architectural change. Only the training objective is changed here. All GPT-based models learn like [2]:

$$P(x_1, x_2, x_3, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1 \ldots x_{i-1}) \tag{5}$$

Here every token learns to predict from the before tokens. Example:

```
def max(a, b):
    if a > b:
        return a
    else:
        return b
```

Now:

```
def max(a, b):
    <MASK>
    if a > b:
        return a
    else:
        return b
```

Heere the model would have the context of `if a > b:` and it does not know `else: return b`. So it may guess `return a`. But it is just a lucky guess, not properly context based. But in InCoder, it would have those contexts and it would be able to generate the code in the middle with much more accuracy.

3.2.1 Model Architecture Overview

The GPT-based architecture model used to see the left context and it would not be able to fill in the middle with much more accuracy. But in Incoder, these things are fixed more or less.

- **Masking the Random Span**
- **Moving the Masked Span to the End**
- **Training the Model to Predict the Masked Span**

4 StructCoder: Structure-aware Code Generation

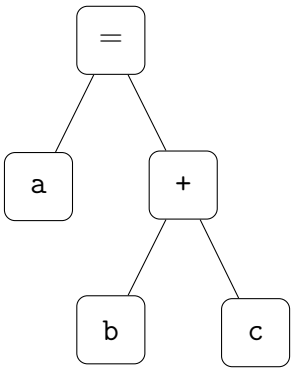
In the transformer-based models, the main problem is that they used to see the code as text. But in reality, the code has its syntax and also the semantics (meaning).

```
a = 5
b = a + 2
```

Here the syntax has to be alright and also the semantics here would be that b is dependent on a. So StructCoder says that code is just not a natural language. Rather the code is a combination of Structure, Dataflow and Syntax. They mainly used two things:

- **Abstract Syntax Tree (AST)**: It is the structure tree of the code.

```
        a = b + c
```



It says which one is operator and which one is variable.

- **Dataflow Graph**: It says which variable is taking value from which variable. Example:

```
a = 5
b = a + 2
```

Here the dataflow graph would be: a → b

StructCoder Architecture: Structcoder = Encoder + Decoder (Transformer-based)

- **Encoder**: It can understand input. The StructCoder encoder uses AST and DFG and Special attention mechanism to understand the structure of the code. It can understand the syntax and semantics of the code.

- **Decoder**: Decoder generates output. Before the decoder used to just predict the next token. But in StructCoder, the decoder is given two extra tasks:

- **AST Path Prediction**: The model is trained to predict the output code's AST path. So it learns the structure of the code.

```
        a = b + c
```

Here model doesn't just predict the code, it also predicts the AST path.

- **Dataflow Prediction**: Model learns which variable depends on which one.

```
        b = a + 2
```

The model can understand a → b

Pre-training Strategy: StructCoder uses Denoising Autoencoder (DAE) as the pre-training strategy. What it does: Code + AST + DFG → it is corrupted.
Then it is asked to the model to recover the original code. Example:

```
    a = b + c
```

After Corruption:

```
    a = __ + c
```

Then the model would fill it with b.

4.1 Structure-Aware Encoder

The structure-aware encoder treats code as more than a simple token sequence. For plain text input, the model can rely only on the surface tokens. When we take code input, it uses the code tokens together with structural information from the AST and DFG.This design follows StructCoder [3] and here it allows the code not only to take the lexical form of the program but also to understand the syntax and semantics through the AST and DFG representations.

Input representation For a code example, the encoder input can be written as follows, following StructCoder [3]:

$$\mathbf{x} = [\text{CLS}, s_1, s_2, \ldots, s_{|S|}, \text{SEP}, l_1, l_2, \ldots, l_{|L|}, v_1, v_2, \ldots, v_{|V|}], \tag{6}$$

where s_i denotes the code tokens, l_i denotes AST leaves, and v_i denotes variables from the DFG. In practice, this means that the encoder can combine token-level information with tree-level and graph-level structure.

For example, the statement `a = b + c` can be represented by the token sequence for the code itself, while the AST and DFG provide additional structure. In a text-only scenario, the model can ignore AST and DFG signals. But in a code scenario, the full structural input is used.

Input embeddings All three input types are mapped into the same vector space. Code tokens use a standard embedding lookup, DFG variables are assigned variable-specific embeddings, and AST leaf embeddings are built from the path between the root and the leaf. StructCoder [3] defines the leaf embedding for a leaf node l as

$$E(l) = \sum_{i=1}^{|l|} E_{\text{type}}(r_i.\text{type}) \odot E_{\text{height}}(|l| - i), \tag{7}$$

where r_i is the i-th node on the path from the root to the leaf. The type embedding captures the node label, while the height embedding helps the model understand the position of the node inside the tree. As a result, the model can learn to recognize patterns in the AST structure.

Structure-aware self-attention The encoder then applies different attention patterns based on the type of relationship being modeled, again following StructCoder [3].

- **Code-to-code attention**: this is the standard token-to-token attention used in transformer models, with relative position information added to preserve order, as described in StructCoder [3].

  $$A(s_i, s_j) = E_{s_i}^\top W_q^\top W_k E_{s_j} + \phi(|i - j|). \tag{8}$$

  Meaning: Standard T5 attention. The relative position function ϕ helps the model understand the order of tokens, which is crucial for code where the sequence matters.

- **Leaf-to-leaf attention**: this attention compares two AST leaves by how much tree context they share. Two leaves that lie on a longer common path are treated as more closely related in StructCoder [3].

$$\mathrm{sim}(l_i, l_j) = \log\left(1 + \frac{\sum_{k=1}^{\min(|l_i|,|l_j|)} \mathbb{1}(r_k^i = r_k^j)}{|l_i||l_j|}\right). \tag{9}$$

The final attention score combines the learned similarity with the normal dot-product attention:

$$A(l_i, l_j) = E_{l_i}^\top W_q^\top W_k E_{l_j} + \big(w_a \cdot \mathrm{sim}(l_i, l_j) + w_b\big). \tag{10}$$

So here it is not based on the position rather is based on the similarity. Example:

```
        = -> + -> b
        = -> + -> c
```

Here the two leaves `b` and `c` share a common path through the AST, so they would have higher attention between them than two leaves that do not share as much structure.

- **Variable-to-variable attention**: in the DFG, attention is allowed only between variables that are connected by a dependency edge, following StructCoder [3].

$$A(v_i, v_j) = \begin{cases} E_{v_i}^\top W_q^\top W_k E_{v_j}, & D_{ij} = 1, \\ -\infty, & \text{otherwise.} \end{cases} \tag{11}$$

Meaning: Only connected variables get the attention here. Example:

```
        b = a + 2
```

Allowed:

```
        a <-> b
```

Not Allowed:

```
        a <-> unrelated variable
```

- **Code-to-leaf and code-to-variable attention**: these cross-type links connect code tokens with the AST leaves and DFG variables that correspond to them, following StructCoder [3].

$$A(s_i, l_j) = \begin{cases} E_{s_i}^\top W_q^\top W_k E_{l_j}, & L_{ij}^{ast} = 1, \\ -\infty, & \text{otherwise.} \end{cases} \tag{12}$$

Meaning: token $\rightarrow$ leaf: if the token corresponds to an AST leaf, then it can attend to that leaf. Example:

```
    a = b + c
```

Links:

```
    a <-> leaf(a)
    b <-> leaf(b)
```

This design gives the encoder a more complete view of the program. Rather than treating the code as a flat sequence of tokens, the model can understand the hierarchical structure of the code through the AST and the dataflow through the DFG. With this attention mechanism, the model can learn to recognize patterns in the code structure.

4.2 Structure-Aware Decoder

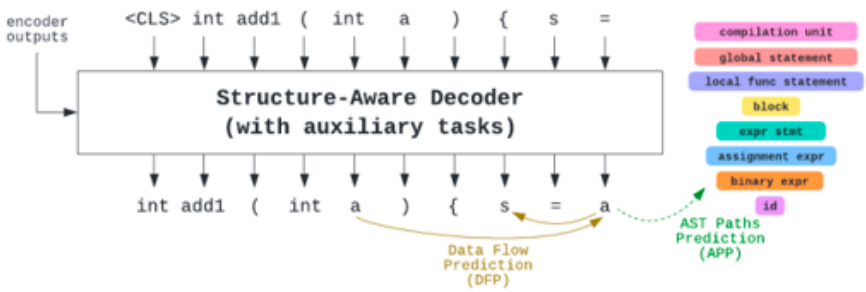


Figure 2: Structure-aware decoder generates the next token in the target code and predicts the node types on the root-to-leaf path to the leaf containing this token in the target AST as well as the DFG edges incident on this token [3].

In normal decoder, it just predicts the next token. But in Structcoder, the decoder does predict next token, ast path and also DFG relations. So it learns the syntax and semantics together.

The interesting thing here is that the output token is not increasing here, rather it is doing extra tasks which helps it in training time. So in inference time, it gets more awareness and prediction becomes better.

Three training tasks StructCoder trains the decoder with three complementary objectives:

- **LM**: next-token prediction.
- **APP**: AST path prediction.
- **DFP**: dataflow prediction.

The language modeling head is the familiar decoder objective. For a hidden state h_i, the probability of the next token is computed as StructCoder [3]:

$$p_i = \text{softmax}(Wh_i). \tag{13}$$

This is the usual transformer-style prediction step: the model takes the hidden representation and turns it into a distribution over the vocabulary. The corresponding loss is also defined in StructCoder [3]:

$$L_{lm} = -\frac{1}{|T|}\sum_{i=1}^{|T|} \log p_i(t_i). \tag{14}$$

This part is actually same as the standard language modeling. So it just ensures that the model learns to generate the correct token sequence overall.

The AST path prediction head is what makes the decoder syntax-aware. For every token, it predicts root→ leaf path. If we take the example `return a + b`, then the token `b` is not just a token by itself; it also sits somewhere in the tree, for example along a path like `=` → `+` → `b`. The decoder learns to recover that path. Formally, the path-level prediction is written as StructCoder [3]:

$$p_{ik}^{ast} = \text{softmax}\Big(W_{ast}^{(|l_i|-k)} h_i\Big). \tag{15}$$

Here the model predicts the k-th node on the path for token i. The loss for this task is defined in StructCoder [3]:

$$L_{app} = -\frac{1}{|T|} \sum_{i=1}^{|T|} \sum_{k=1}^{|l_i|} \log p_{ik}^{ast}(r_k^i.\text{type}). \tag{16}$$

In intuitive words, here the model learns where to place which token in tree. So it results in syntax-aware generation.

The third task is dataflow prediction. Here model gets to understand the semantic relationships. For every pair of tokens (i, j), the decoder predicts if there is a dataflow link from token j to token i. For example, in `a = b + c`, both `b` and `c` influence `a`. So the model should learn that `b` → `a` and `c` → `a`. StructCoder models this relation as

$$p_{ij}^{dfg} = \sigma\big(h_i^\top U_{dfg}^\top V_{dfg} h_j + u_{dfg}^\top h_i + v_{dfg}^\top h_j + w_{dfg}\big). \tag{17}$$

The sigmoid makes the result a probability, so the model can decide whether the edge exists or not. It also impiles that direction matters. The corresponding binary cross-entropy loss is also from StructCoder [3]:

$$L_{dfp} = -\frac{1}{|T|^2} \sum_{i=1}^{|T|} \sum_{j=1}^{|T|} \Big[y_{ij} \log p_{ij}^{dfg} + (1 - y_{ij}) \log(1 - p_{ij}^{dfg}) \Big]. \tag{18}$$

It eventually helps in semantic-aware generation.

Final objective StructCoder combines the three losses into one final objective [3]:

$$L = L_{lm} + \alpha_1 L_{app} + \alpha_2 L_{dfp}. \tag{19}$$

Here L_{lm} controls token correctness, L_{app} controls syntax correctness, and L_{dfp} controls semantic correctness. The weights α_1 and α_2 balance how much the syntax and dataflow tasks matter during training. So the decoder is still generating code normally, but it learns from AST and DFG supervision at the same time [3].

For example, if the model sees `a = b + c`, the language modeling head learns to generate the token sequence, the AST path head learns the tree structure of each token, and the DFG head learns that `b` and `c` flow into `a`. It is the core idea behind this Structure-aware decoder.

5 Draft Structure

This section can be replaced with the actual content of the paper. A typical technical paper in this format can include a problem statement, methodology, experiments, results, and discussion. Keeping the layout in one column makes it easier to write, review, and annotate while preserving IEEE-style citations.

6 Conclusion

The document is now set up as a single-column LaTeX file with IEEE numeric references. You can expand each section as needed without changing the layout or citation style.

References

[1] Y. Wang, W. Wang, S. R. Joty, and S. C. H. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *CoRR*, vol. abs/2109.00859, 2021. [Online]. Available: https://arxiv.org/abs/2109.00859

[2] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W. tau Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," 2023. [Online]. Available: https://arxiv.org/abs/2204.05999

[3] S. Tipirneni, M. Zhu, and C. K. Reddy, "Structcoder: Structure-aware transformer for code generation," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 3, Jan. 2024. [Online]. Available: https://doi.org/10.1145/3636430

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[5] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," 2020. [Online]. Available: https://arxiv.org/abs/2002.08155

[6] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "Graphcodebert: Pre-training code representations with data flow," 2021. [Online]. Available: https://arxiv.org/abs/2009.08366

[7] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," 2021. [Online]. Available: https://arxiv.org/abs/2103.06333